Terminal Linux — Tema 2: Permisos, Procesos y Pipes root@bash:~#

Nivel 2: aquí ya no eres turista en la terminal. Esto es lo que se usa a diario en cualquier servidor real: quién puede tocar qué, qué está corriendo, y cómo encadenar comandos como un profesional.

1. Permisos en profundidad: chmod y chown

Ya sabes leer `rwxrwxrwx` (Propietario/Grupo/Otros). Ahora vas a CAMBIARLOS. Cada permiso tiene un valor numérico: **r=4, w=2, x=1**. Sumas los que quieras por bloque.

```
chmod 755 script.sh
→ rwx r-x r-x (7=4+2+1, 5=4+1, 5=4+1)
chmod 644 archivo.txt
→ rw- r-- r-- (6=4+2, 4=4, 4=4)
```

Valor	Permiso
7	rwx (leer+escribir+ejecutar)
6	rw- (leer+escribir)
5	r-x (leer+ejecutar)
4	r-- (solo leer)
0	--- (nada)

Modo simbólico (alternativa a los números): `chmod +x script.sh` añade ejecución a todos. `chmod u+w archivo` añade escritura solo al propietario (u=user, g=group, o=others, a=all).

`chown usuario:grupo archivo` → cambia el DUEÑO y el grupo de un archivo. `chown -R usuario:grupo carpeta/` → lo aplica a toda la carpeta y su contenido (recursivo, igual que `cp -r`).

TRUCO Nº5: `chmod +x` es el comando más usado del mundo real. Si descargas un script y da "Permission denied" al ejecutarlo, el 90% de las veces es porque le falta permiso de ejecución. Solución: `chmod +x nombre.sh`.

⚠ **NUNCA hagas `chmod 777` en un servidor real. Da permiso total (leer/escribir/ejecutar) a CUALQUIERA. Es la primera cosa que busca un atacante al escanear un sistema.**

2. Usuarios y superusuario

Comando	Uso
`whoami`	Qué usuario eres ahora mismo
`id`	Tu UID, GID y grupos a los que perteneces
`su usuario`	Cambia de usuario (pide contraseña)
`sudo comando`	Ejecuta UN comando como root, sin cambiar de sesión
`sudo -i`	Abre una sesión completa como root
`passwd`	Cambia tu contraseña
`adduser nombre`	Crea un usuario nuevo (requiere sudo)

root vs sudo: `root` es el superusuario, puede hacer literalmente cualquier cosa al sistema. En Ubuntu no inicias sesión como root directamente por seguridad; usas `sudo` para pedir permiso puntual. Cada uso de `sudo` queda registrado en `/var/log/auth.log`.

3. Procesos: qué está corriendo

Comando	Uso
`ps`	Procesos de TU terminal actual
`ps aux`	TODOS los procesos del sistema, de todos los usuarios
`top`	Monitor en vivo: CPU, RAM, procesos (actualiza solo)
`htop`	Como `top` pero más visual (si está instalado)
`kill PID`	Termina un proceso por su número (PID)
`kill -9 PID`	Lo mata a la fuerza, sin piedad (último recurso)
`killall nombre`	Mata todos los procesos con ese nombre
`comando &`	Lo lanza en segundo plano y te devuelve la terminal
`jobs`	Lista procesos en segundo plano de tu sesión
`fg`	Trae de vuelta a primer plano el último proceso en 2º plano

TRUCO Nº6: en `top` / `htop`, pulsa `q` para salir. Fíjate en la columna `%CPU` y `%MEM`: si algo dispara esas cifras, ahí está tu problema de rendimiento. `Ctrl+Z` pausa un proceso en marcha (no lo mata) y `bg` lo manda a segundo plano.

4. Redirección: dónde va la salida

Símbolo	Qué hace
`>`	Guarda la salida en un archivo, SOBRESCRIBE lo que había
`>>`	Añade la salida al final del archivo, sin borrar nada
`<`	Usa un archivo como entrada de un comando
`2>`	Redirige solo los mensajes de error a un archivo
`&>`	Redirige salida normal Y errores juntos

```
ls -al > listado.txt # crea/sobrescribe
echo "nota" >> diario.txt # añade al final
comando 2> errores.log # solo errores
```

⚠ **`>` sobrescribe sin avisar ni preguntar. Si usas `>` en vez de `>>` por error, pierdes el contenido anterior para siempre (recuerda: no hay papelera).**

5. Pipes y búsqueda de texto: el combo que lo cambia todo

El pipe `|` conecta comandos: la SALIDA del primero se convierte en la ENTRADA del segundo. Así es como los profesionales encadenan herramientas simples para hacer cosas complejas, en vez de memorizar comandos gigantes.

Comando	Uso
`grep "texto" archivo`	Busca líneas que contienen "texto" dentro de un archivo
`grep -i "texto" archivo`	Igual, pero ignora mayúsculas/minúsculas
`grep -r "texto" carpeta/`	Busca en TODOS los archivos de una carpeta (recursivo)
`grep -n "texto" archivo`	Muestra también el número de línea donde aparece
`find carpeta -name "*.txt"`	Busca archivos por nombre/patrón dentro de una carpeta
`find . -type d`	Busca solo carpetas (`-type f` = solo archivos)
`comando | grep algo`	Filtra la salida de cualquier comando por texto
`comando | wc -l`	Cuenta cuántas líneas devuelve un comando
`comando | sort`	Ordena alfabéticamente la salida
`comando | head -n 5`	Muestra solo las primeras 5 líneas de la salida

`comando | tail -n 5`	Muestra solo las últimas 5 líneas
`tail -f archivo.log`	Muestra un log EN VIVO, según se va escribiendo (clave para depurar servidores)
`cat archivo`	Muestra todo el contenido de un archivo de golpe
`less archivo`	Muestra el contenido página a página (`q` para salir, `/palabra` para buscar dentro)

TRUCO Nº7 (combo real de sysadmin): `ps aux | grep nombre_programa` te encuentra el PID de un proceso concreto en segundos, sin leer toda la lista. Es el combo más usado del día a día.

TRUCO Nº8: `history | grep comando` busca en tu historial sin tener que hacer scroll infinito. Combínalo con `Ctrl+R` (que ya viste en el Tema 1) y nunca más reescribirás un comando largo dos veces.

Diferencia clave grep vs find: `grep` busca DENTRO del contenido de archivos (texto). `find` busca archivos/carpetas por su NOMBRE, tipo o fecha, sin mirar su contenido. No los confundas: es el error nº1 de los que empiezan.

6. Paquetes: instalar y actualizar software (APT)

Comando	Uso
`sudo apt update`	Actualiza la lista de paquetes disponibles (no instala nada aún)
`sudo apt upgrade`	Instala las actualizaciones disponibles de lo que ya tienes
`sudo apt install paquete`	Instala un programa nuevo
`sudo apt remove paquete`	Desinstala un programa
`apt search palabra`	Busca paquetes disponibles por nombre

Regla: siempre `update` antes de `install` o `upgrade` — si no, puedes instalar versiones desactualizadas o fallar por no conocer el paquete más reciente.

7. Variables de entorno y PATH

`echo $PATH` muestra las carpetas donde el sistema busca los comandos que escribes (por eso `ls` funciona desde cualquier sitio: está en una carpeta del PATH). `echo $HOME` muestra tu carpeta personal. `export VARIABLE=valor` crea una variable de entorno temporal para tu sesión.

MENTALIDAD PRO Nivel 2: ya no piensas en comandos sueltos, piensas en TUBERÍAS de datos: algo genera información (`ls` , `ps` , `cat`) → algo la filtra (`grep`) → algo la cuenta u ordena (`wc` , `sort`) → algo la guarda (`>`). Ese es el flujo mental de cualquier admin de sistemas hoy en día, da igual el proveedor cloud o la distro.

ORDEN DE APRENDIZAJE Tema 2: 1) `chmod` / `chown` con números (7-6-5-4) → 2) `ps aux` y `kill` → 3) `>` vs `>>` → 4) `grep` y `find` por separado → 5) combínalos con pipes `|` → 6) `apt` para gestionar software. Practica cada bloque en tu terminal real, no solo leas.

Regla de oro Nivel 2: antes de un `kill -9` o un `chmod 777` , pregúntate qué se rompe si te equivocas. En producción, "probar a ver qué pasa" puede tirar un servidor entero.